大模型部署与应用实验任务书

先说两句

这份任务书接着 Rocky Linux 基础操作实验往下走。前面练的 `cd`、`vim`、`dnf`，还有编译那一套，这回要真刀真枪地用上了。

一共五个实验。先在 Linux 上把一个多模态大模型跑起来，量化之后测测性能差多少；然后转到 Windows 上折腾提示词工程；再自己动手写个聊天程序；接着去了解 MCP 和 Skill 这两个新东西；最后用 Docker 把模型重新部署一遍，体会一下什么叫"一条命令的事"。

遇到问题照旧问 AI。不过这次你手上有自己部署的模型了，同一个问题也可以拿去问问它，感受一下 2B 小模型和云端大模型差多远。

实验前确认

- [ ] Rocky Linux 基础操作的 7 个实验已完成
- [ ] 虚拟机内存已调到 8G，磁盘剩余空间 ≥ 20G（`free -h` 和 `df -Th` 确认）
- [ ] 已按照《03.大模型本地部署》文档编译好 `llama.cpp`（`ls llama.cpp/build/bin` 能看到一堆命令）
- [ ] 已创建并会激活 python 虚拟环境（`source .venv/bin/activate`）
- [ ] Windows 宿主机上下载好 LM Studio 和 Cherry Studio 的安装包
- [ ] 拍了虚拟机快照
- [ ] 有一个能用的 AI 工具

AI 怎么用

你遇到的情况	怎么做
命令报错	完整复制报错信息贴给 AI 分析
概念不懂	问："用通俗的话解释什么是模型量化"
参数不会填	问："llama-bench 的 -p 和 -n 参数是什么意思？"
代码跑不通	把代码和报错一起贴给 AI
下载慢、连不上	问 AI 有没有国内镜像，或者直接问老师

> AI 说的不一定对，工具类的知识更新太快，它的训练数据经常是过时的。以官方文档和你自己的验证为准。

实验一：部署 Qwen3-VL-2B，测测量化到底损失了什么

你要学会的

- 从 ModelScope 下载模型原始权重
- 用 `convert_hf_to_gguf.py` 把权重转成 GGUF 格式（包括视觉投影器）
- 用 `llama-quantize` 做 Q8_0 和 Q4_K_M 量化
- 用 `llama-bench` 对比不同量化级别的推理性能
- 用 `llama-server` 把模型部署成 OpenAI 兼容的 API 服务

先问 AI

问："大模型量化里的 BF16、Q8_0、Q4_K_M 分别是什么意思？量化会带来什么代价？"

```
AI 说的（摘要）：


```

动手敲

第一步：下载模型

这次用 `Qwen/Qwen3-VL-2B-Instruct`。2B 参数，个头小，虚拟机跑得动，而且能看图。

```
source .venv/bin/activate
# 先激活虚拟环境，命令行前面出现 (.venv) 才算成功

pip install modelscope
# 之前装过就跳过

modelscope download --model Qwen/Qwen3-VL-2B-Instruct --local_dir ./Qwen3-VL-2B-Instruct
# 下载模型原始权重到当前目录的 Qwen3-VL-2B-Instruct 目录

ls -lh Qwen3-VL-2B-Instruct
# 看看下载了什么，注意 safetensors 文件的大小
```

第二步：转换为 GGUF 格式

```
python llama.cpp/convert_hf_to_gguf.py ./Qwen3-VL-2B-Instruct --outtype bf16
# 转换语言模型部分，得到 BF16 精度的 GGUF 文件

python llama.cpp/convert_hf_to_gguf.py ./Qwen3-VL-2B-Instruct --mmproj
# 这是个视觉模型，还要单独转换视觉投影器
# 会得到一个 mmproj- 开头的 GGUF 文件

ls -lh Qwen3-VL-2B-Instruct/*.gguf
# 确认两个 gguf 文件都在，记下确切的文件名，后面命令要用
```

第三步：量化出 Q8_0 和 Q4_K_M 两个版本

```
./llama.cpp/build/bin/llama-quantize \
  Qwen3-VL-2B-Instruct/Qwen3-VL-2B-Instruct-BF16.gguf \
  Qwen3-VL-2B-Instruct/Qwen3-VL-2B-Instruct-Q8_0.gguf Q8_0
# BF16 -> Q8_0

./llama.cpp/build/bin/llama-quantize \
  Qwen3-VL-2B-Instruct/Qwen3-VL-2B-Instruct-BF16.gguf \
  Qwen3-VL-2B-Instruct/Qwen3-VL-2B-Instruct-Q4_K_M.gguf Q4_K_M
# BF16 -> Q4_K_M

ls -lh Qwen3-VL-2B-Instruct/*.gguf
# 对比一下三个精度的文件大小
```

把三个文件的大小记下来：

量化级别	文件大小	大约是 BF16 的几分之几
BF16		1
Q8_0		
Q4_K_M		

第四步：用 llama-bench 跑性能测试

`llama-bench` 是 llama.cpp 自带的基准测试工具，默认测两项：

- `pp512`：处理 512 个 token 的提示词有多快（提示词处理速度）
- `tg128`：连续生成 128 个 token 有多快（文字生成速度）

单位都是 t/s，每秒 token 数，越大越好。

```
./llama.cpp/build/bin/llama-bench \
  -m Qwen3-VL-2B-Instruct/Qwen3-VL-2B-Instruct-BF16.gguf \
  -m Qwen3-VL-2B-Instruct/Qwen3-VL-2B-Instruct-Q8_0.gguf \
  -m Qwen3-VL-2B-Instruct/Qwen3-VL-2B-Instruct-Q4_K_M.gguf
# 一条命令测三个模型，要等不少时间，去倒杯水
```

记一下输出

量化级别	pp512 (t/s)	tg128 (t/s)
BF16		
Q8_0		
Q4_K_M		

第五步：把 Q4_K_M 版本部署成服务

```
systemctl disable --now firewalld
# 简化实验，先关掉防火墙

./llama.cpp/build/bin/llama-server \
  -m Qwen3-VL-2B-Instruct/Qwen3-VL-2B-Instruct-Q4_K_M.gguf \
  -mm Qwen3-VL-2B-Instruct/mmproj-Qwen3-VL-2B-Instruct-BF16.gguf \
  --host 0.0.0.0 --port 8080 -a Qwen3-VL-2B
# -mm 指定视觉投影器，这样模型才能"看图"
# -a 指定对外的模型名字
# mmproj 文件名以你第二步 ls 看到的为准
```

在 Windows 浏览器里访问 `http://虚拟机IP:8080`（虚拟机 IP 用 `ip addr` 查），先聊几句，再发一张图片让它描述内容，看看视觉部分工作没有。

想一想

1. Q4_K_M 的文件只有 BF16 的三分之一左右，tg128 速度快了多少？为什么量化能加速推理？
2. 找同一个问题分别问 BF16 和 Q4_K_M 版本（用 `llama-cli -m xxx.gguf` 就行），你能感受到"降智"吗？
3. 为什么视觉投影器 mmproj 一般不做激进量化？

```
先自己想、动手试，再问 AI 确认：


```

实验二：Cherry Studio 里玩提示词工程

你要学会的

- 在 Windows 上用 LM Studio 部署模型，并开启 GPU 加速
- 把 LM Studio 的 API 服务接入 Cherry Studio
- 拿几种常见的提示词技巧逐个做对照实验，看它们到底管不管用

先问 AI

问："什么是提示词工程？system prompt、few-shot、chain-of-thought 分别是什么？"

```
AI 说的（摘要）：


```

动手做

第一步：LM Studio 部署模型 + GPU 加速

1. 安装 LM Studio（记得启用开发者模式），在里面搜索并下载一个 4B 左右的模型，比如 Qwen 系列，选 Q4_K_M 版本
2. 加载模型时注意 GPU Offload 设置，把尽量多的层放到 GPU 上
3. 打开 Windows 任务管理器的 GPU 页面，跟模型对话时观察 GPU 占用有没有上去
4. 在 LM Studio 的开发者（Developer）页面启动 API Server，默认地址 `http://127.0.0.1:1234`

> 做个对比：把 GPU Offload 拉到 0（纯 CPU）和拉满，同一个问题各问一次，生成速度差多少？LM Studio 会显示 tok/sec，记下来。

模式	生成速度 (tok/s)
纯 CPU	
GPU 加速	

第二步：Cherry Studio 接入

1. 安装 Cherry Studio，进 设置 → 模型服务，找到 LM Studio（没有就手动添加一个 OpenAI 兼容的服务商）
2. API 地址填 `http://127.0.0.1:1234`，然后添加你刚才加载的模型
3. 回到对话页面选中这个模型，随便聊一句，通了再往下做

第三步：四组提示词对照实验

每组都是"不用技巧"和"用了技巧"各问一次，对比输出差异。系统提示词在 Cherry Studio 的助手设置里改。

实验 A：系统提示词 / 角色设定

轮次	系统提示词	用户问题
对照	（留空）	给我推荐三本书
实验	你是一位只用文言文回答问题的古代私塾先生	给我推荐三本书

实验 B：零样本 vs 少样本（few-shot）

轮次	提示词
对照	判断这句话的情感：这家店的菜量倒是挺大的，就是味道嘛……
实验	判断情感，只输出"正面"或"负面"。例子：「服务很贴心」→正面；「排队两小时，态度还差」→负面。 现在判断：这家店的菜量倒是挺大的，就是味道嘛……

实验 C：思维链（chain-of-thought）

轮次	提示词
对照	鸡兔同笼，头 35 个，脚 94 只，鸡兔各几只？直接给出答案，不要过程。
实验	鸡兔同笼，头 35 个，脚 94 只，鸡兔各几只？请一步一步推理，最后给出答案。

实验 D：输出格式约束

轮次	提示词
对照	介绍一下南京这座城市
实验	介绍南京，严格按 JSON 格式输出，字段：{"city": "", "province": "", "highlights": ["", "", ""]}，不要输出 JSON 以外的任何内容

记一下结果

实验	对照组输出特点	实验组输出特点	技巧有效吗
A 角色设定			
B 少样本			
C 思维链			
D 格式约束			

想一想

1. 实验 C 里，不让模型写过程直接给答案，它更容易算错。为什么"让模型多说话"反而更准？
2. 把助手设置里的 temperature 调到 0.1 和 1.2，同一个问题各问三遍，输出的稳定性有什么区别？

```
我的观察和理解：


```

实验三：用 Python + openai 库写自己的聊天程序

你要学会的

- openai 库的基本用法：`base_url`、`api_key`、`chat.completions.create`
- 什么是 messages 列表，多轮对话的上下文是怎么维护的
- 写出一个能连续对话的 CLI 程序

先问 AI

问："OpenAI 兼容 API 是什么意思？为什么本地部署的 llama-server 和 LM Studio 都能用 openai 这个库调用？"

```
AI 说的（摘要）：


```

动手敲

在哪台机器上写都行，Linux 虚拟机里用 vim，或者 Windows 上用你顺手的编辑器。API 地址二选一：

- 实验一里 Linux 上的 llama-server：`http://虚拟机IP:8080/v1`
- 实验二里 Windows 上的 LM Studio：`http://127.0.0.1:1234/v1`

```
pip install openai
# 记得在虚拟环境里装
```

第一步：最小可用示例

先跑通一次性的调用，新建 `hello_llm.py`：

```python
from openai import OpenAI

client = OpenAI(
    base_url="http://127.0.0.1:8080/v1",  # 改成你的实际地址
    api_key="none",  # 本地部署不校验 key，随便填但不能不填
)

resp = client.chat.completions.create(
    model="Qwen3-VL-2B",  # 就是 llama-server 里 -a 指定的名字
    messages=[
        {"role": "user", "content": "用一句话介绍你自己"},
    ],
)
print(resp.choices[0].message.content)
```

```
python hello_llm.py
```

第二步：升级成多轮对话 CLI

新建 `chat_cli.py`。关键在于把每一轮的对话都追加进 `messages`，模型才有"记忆"：

```python
from openai import OpenAI

client = OpenAI(base_url="http://127.0.0.1:8080/v1", api_key="none")

messages = [
    {"role": "system", "content": "你是一个简洁友好的中文助手"},
```

```
]

print("开始对话，输入 exit 退出")
while True:
    user_input = input("\n你: ").strip()
    if user_input in ("exit", "quit"):
        break
    if not user_input:
        continue

    messages.append({"role": "user", "content": user_input})
    resp = client.chat.completions.create(
        model="Qwen3-VL-2B",
        messages=messages,
    )
    reply = resp.choices[0].message.content
    print(f"AI: {reply}")
    messages.append({"role": "assistant", "content": reply})
```

跑起来之后做个验证：先告诉它"我叫小王"，隔两轮再问"我叫什么？"它答得上来，说明你的上下文维护是对的。

第三步（选做，任选其一）

- 流式输出：给 `create()` 加 `stream=True`，让回答像打字机一样逐字蹦出来。不会写就问 AI："openai 库 stream=True 怎么逐块打印内容？"
- GUI 界面：`pip install gradio`，用 `gr.ChatInterface` 十几行代码就能做出网页聊天界面。问 AI 要一个最小示例，自己改通。

记一下

```
【贴上你的程序运行截图说明：多轮对话记忆验证是否成功】


```

想一想

1. `messages` 列表会越聊越长，而模型的上下文窗口是有限的。聊得太久会发生什么？有什么解决思路？
2. system 角色的消息和 user 角色的消息，对模型行为的影响有什么不同？

```
我的理解（可以动手验证）：


```

实验四：认识 MCP 和 Skill——大模型的"外挂"生态

你要学会的

- MCP（Model Context Protocol）是什么，解决了什么问题
- Skill 是什么，和 MCP 有什么区别
- 自己查文档，收集和评估这些生态工具的能力

先问 AI

问："MCP（Model Context Protocol）是什么？它和大模型的 Function Calling 是什么关系？"

再问："Anthropic 提出的 Agent Skill 是什么？和 MCP 的定位有什么不同？"

```
AI 说的（摘要）：


```

动手做

第一步：读资料

- MCP 官方文档：https://modelcontextprotocol.io
- MCP 官方服务器仓库：https://github.com/modelcontextprotocol/servers
- MCP 导航站（找现成 MCP 用）：https://mcp.so
- Agent Skills 仓库：https://github.com/anthropics/skills

> 打不开的就问 AI，或者搜国内的介绍文章。注意看文章的发布时间，这个领域半年前的文章可能已经过时了。

第二步：收集并填表

至少收集 3 个 MCP 服务器和 2 个 Skill，整理成表：

类型	名称	作用（一两句话说清）	你觉得适合什么场景
MCP			
MCP			
MCP			
Skill			
Skill			

第三步：在 Cherry Studio 里实际配一个 MCP

Cherry Studio 支持 MCP：设置 → MCP 服务器。第一次使用会提示安装 uv / bun 运行环境，按提示装好。

推荐从简单的入手，比如 `fetch`（让模型能抓网页）或 `time`（让模型知道现在几点）。配置好后，在对话中开启这个 MCP，试着问：

- 配了 `time`：问"现在北京时间几点了？"
- 配了 `fetch`：问"帮我抓取并总结这个网页的内容：<某个网址>"

然后关掉 MCP 再问一遍同样的问题，看模型怎么回答。

记一下

```
【MCP 调用成功的对话截图说明 + 开关 MCP 前后回答的差异】


```

想一想

1. 大模型本身"知道"现在几点吗？为什么需要 MCP 这样的机制？
2. 给模型接上能读写文件、能执行命令的 MCP，方便是方便了，安全上要担心什么？

```
我的思考：



```

实验五：用 Docker 容器一条命令部署模型

你要学会的

- 在 Rocky Linux 上安装 Docker
- `docker pull / tag / run / ps / logs / stop / rm` 这套基本操作
- 用 llama.cpp 官方容器镜像部署模型，和实验一的手工编译比较一下

先问 AI

问："Docker 是什么？镜像和容器是什么关系？和虚拟机有什么区别？"

```
AI 说的（摘要）：


```

动手敲

第一步：安装 Docker

```
curl -o /etc/yum.repos.d/docker-ce.repo https://mirrors.aliyun.com/docker-ce/linux/centos/docker-ce.repo
# 添加阿里云的 docker-ce 软件仓库，国内下载快

dnf install docker-ce docker-ce-cli containerd.io -y
# 安装 docker

systemctl enable --now docker
# 启动 docker 服务并设置开机自启

docker --version
docker info | head -20
# 验证安装成功
```

> 这一步如果报错（不同 Rocky 版本仓库配置可能有差异），把报错完整贴给 AI 分析。

第二步：拉取 llama.cpp 服务端镜像

llama.cpp 官方镜像放在 GitHub 的容器仓库（ghcr.io）上，国内直接拉大概率超时。解决办法是走公共镜像代理 `1ms.run`：在原本的镜像名前面加上代理地址当前缀，就能从国内节点拉取。

```
docker pull ghcr.1ms.run/ghcr.io/ggml-org/llama.cpp:server
# 通过 1ms.run 代理拉取 ghcr.io 上的官方镜像

docker images
# 看看拉下来的镜像，名字前面挂着一长串代理地址，后面写命令很啰嗦
```

> 如果上面的 `docker pull` 一直卡住或者报超时——正常，国内网络访问外网仓库翻车很常见。准备了一个备用方案：把镜像提前打包成 tar 文件，直接下载回来灌进 docker。

备用方案：tar 包离线载入

```
curl -L -o ./llama.cpp.server.tar https://file.boulderh.top/llama.cpp.server.tar
# 下载打好包的镜像文件，大概 3 个多 G，耐心等

docker load -i ./llama.cpp.server.tar
# docker load 把 tar 包里的镜像还原到本地
# 这一步就跟 docker pull 效果一样，只不过不走网络拉取，读本地文件

docker images
# 确认镜像已经出现在列表里
```

两种方式（pull 和 load）殊途同归，最终本地都会有一个 `ghcr.io/ggml-org/llama.cpp:server` 镜像。如果 tar 包里出来的镜像名字和文中的不一样，用 `docker images` 先看清楚实际名字，往后用那个就行。

如果用的是 docker pull 拉回来的，镜像名前面挂着 `docker.1ms.run/` 前缀，`docker tag` 可以给它换个短名。tag 只是多加了一个名字，镜像本体只有一份：

```
docker tag ghcr.1ms.run/ggml-org/llama.cpp:server ghcr.io/ggml-org/llama.cpp:server
# 给镜像挂上官方原名

docker rmi ghcr.1ms.run/ghcr.io/ggml-org/llama.cpp:server
# 把带代理前缀的旧名字摘掉（删的只是名字，镜像还在）

docker images
# 确认现在镜像叫 ghcr.io/ggml-org/llama.cpp:server，IMAGE ID 和删名前是同一个
```

第三步：运行容器部署模型

直接复用实验一量化好的 Q4_K_M 模型文件，不用重新下载：

```
docker run -d --name llama-server \
  -p 8080:8080 \
  -v ~/Qwen3-VL-2B-Instruct:/models \
  ghcr.io/ggml-org/llama.cpp:server \
  -m /models/Qwen3-VL-2B-Instruct-Q4_K_M.gguf \
  --host 0.0.0.0 --port 8080 -a Qwen3-VL-2B
# -d 后台运行
# --name 给容器起个名字
# -p 8080:8080 把容器里的 8080 端口映射到虚拟机的 8080 端口
# -v 把宿主机的模型目录挂载到容器内的 /models
# 注意：运行前先确认实验一的 llama-server 已经停掉了，否则端口冲突
```

第四步：验证和管理

```
docker ps
# 看容器是不是在跑（STATUS 一列是 Up）

docker logs llama-server
# 看容器里模型加载的日志

curl http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model":"Qwen3-VL-2B","messages":[{"role":"user","content":"你好，用一句话介绍你自己"}]}'
# 用 curl 直接测 API，也可以照旧用 Windows 浏览器访问

docker stop llama-server
# 停掉容器

docker ps -a
# 停掉的容器还在列表里（STATUS 变成 Exited）

docker start llama-server
# 再拉起来，注意这次模型加载有多快

docker rm -f llama-server
# 实验做完，彻底删掉容器（镜像还在，随时能再 run 一个）
```

记一下

```
【docker ps 和 curl 测试成功的输出贴这里】


```

想一想

1. 实验一我们编译 llama.cpp 花了大半天，这次 docker 几分钟就部署完了。那编译那套流程还有存在的意义吗？什么场景下必须自己编译？
2. 容器删掉之后，挂载在 `-v` 目录里的模型文件还在吗？为什么？动手验证。
3. Rocky Linux 其实自带一个叫 `podman` 的工具，问问 AI 它和 Docker 是什么关系。

```
先自己想、动手验证，再问 AI：



```

最后交什么

对照下面的清单检查，整理进实验报告：

实验	必交内容
一	三个 GGUF 文件大小表、llama-bench 性能表、llama-server 网页对话 + 图片理解截图
二	CPU/GPU 速度对比表、四组提示词对照实验记录表
三	聊天程序源码 + 多轮记忆验证的运行截图
四	MCP / Skill 收集表、Cherry Studio 中 MCP 调用成功截图
五	docker tag 改名前后的镜像列表、docker ps + curl 测试截图

每个实验的"想一想"都要写，写你自己的理解。别整段复制 AI 的回答，看得出来的。